Learning from Failure: Inference-Time Self-Improvement for Computer-Use Agents

Xueqiao Sun, Xiaohan Wang, Ludwig Schmidt, Serena Yeung-Levy, Yuhui Zhang

Paper version: arXiv 2606.31270v1 · CUA Radar Reel poster, not made or endorsed by the authors

Problem

> *all failed trajectories are discarded, despite the substantial cost of building verifiable environments*

Success-based loops fine-tune on wins; **this paper mines the failures** into lightly human-verified inference-time patches.

Motivation: failures are wasted

1. **Success-only SFT.** Wins kept, failures tossed.
2. **Sunk cost.** Environments took engineering.
3. **The bet.** Failures name weaknesses; patches fix.

Evaluation setup

OSWorld · 100 steps | OpenCUA-72B base | 3 runs, mean ± std | OSWorld-small · 30 steps | Claude 4.5 Sonnet judge

- **Online** OSWorld success; ablation on a 30-step small set.
- Patches mined on OSWorld, moved to 4 other benchmarks.

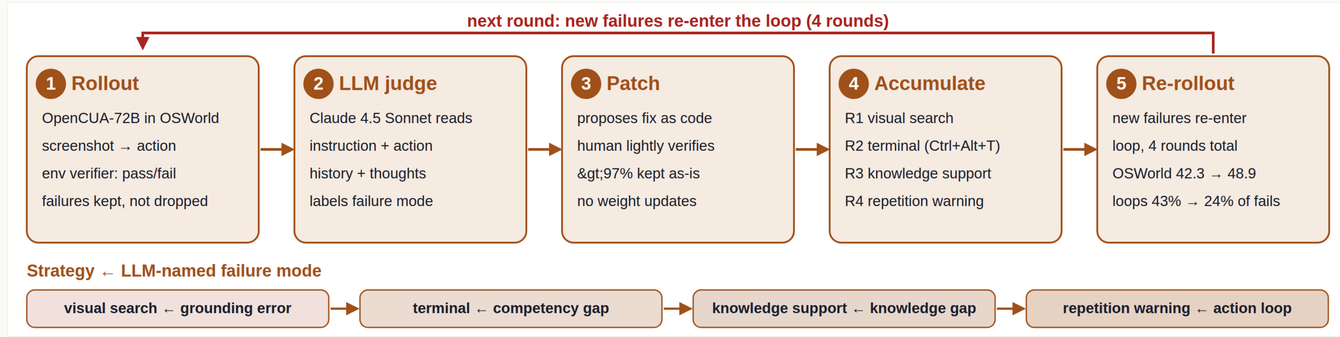
Our read (CUA Radar)

61	★★	transfer
score of 100	band	evidence level

- **Reports:** online OSWorld eval; 3 runs ± std; cost; per-strategy ablation; cross-model and cross-benchmark transfer. **Not reported:** code unclear.
- **Flags:** AP5 — patches mined and evaluated on OSWorld; baselines at mixed step budgets.
- Pattern C19. Authors' numbers, not rerun.

Method: a failure-case loop around a frozen agent

- **Sees / acts:** screenshots in; clicks, hotkeys, terminal, search out.
- **Loop:** LLM judge reads failed trajectories, proposes code patches; human lightly verifies; 4 rounds.
- **Signal:** environment verifier labels each rollout; failures, not successes, drive the patches.



Our redraw (not the paper's figure): the four-round failure-case loop around OpenCUA-72B in OSWorld; strategies accumulate across rounds.

$$\text{warn} \iff n(p) \geq 3 \text{ in a 5-step window} \ \vee \ \text{hash(screen) unchanged for 3 steps}$$

repetition-warning rule (Appendix A.5); a warning switches the agent to recovery mode

Key results: OSWorld success rate (%)

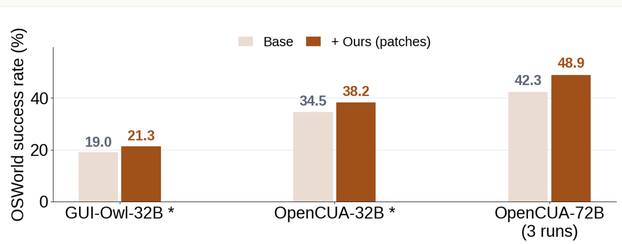


Table 2, redrawn. 72B rows: mean of 3 runs (± 2.6 / ± 1.2); 32B and GUI-Owl: single runs (*).

+6.6 pts (42.3 → 48.9), no training cost, ~8% runtime overhead

SO WHAT → ** rows are single runs (Table 2); all rows share the 100-step budget and action space.*

Ablation: OSWorld-small (%)

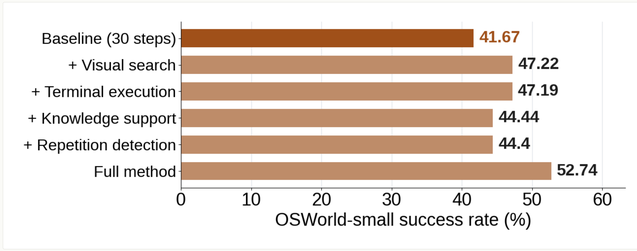


Table 1, redrawn: OSWorld small set, 30-step limit; each strategy applied alone to the same base agent, versus all four combined.

SO WHAT → *Every strategy helps alone — the text states +5.52 for terminal execution and +2.73 for repetition detection — and combining all four reaches 52.74, above any single strategy (baseline 41.67). The failure-mode mix also shifts: action loops fall from 43% to 24% of failures, while multi-step planning (28%) and UI recognition (24%) dominate what remains.*

Headline Numbers

48.9%
OSWorld success rate
vs 42.3 base OpenCUA-72B

- **+6.6** pts, no training
- **52.74%** small set, all patches
- **+7.86** AndroidControl, offline

The paper's own controls

CONTROL	RESULT	CONTEXT
Tools / env / actions	identical to baseline	no extra access
Overhead	+8% runtime, −15% steps	zero extra training
Human verification	>97% kept as-is	<3% lines adjusted
Best prior	o3 42.9 @ 200 steps	ours 48.9 @ 100 steps

Takeaway

LLM-diagnosed failures become four inference-time patches: 42.3 → 48.9 on OSWorld with no training; the recipe transfers across models.

+6.6 pts on OSWorld | >97% patches kept | loops 43% → 24%

Scan to read the full paper

arxiv.org/abs/2606.31270v1